7 Funzioni

Questa dispensa propone esercizi sulla scrittura di algoritmi, in linguaggio C, utili alla comprensione della definizione e dell'utilizzo delle **funzioni**.

Le funzioni sono blocchi di codice riutilizzabili che eseguono un compito specifico. Permettono di organizzare il programma in parti più piccole e gestibili (principio di *modularità*), migliorando la leggibilità e riducendo la duplicazione del codice.

Un programma C è composto da una o più funzioni; la funzione `main()` è obbligatoria ed è il punto di ingresso del programma.

7.1 Dichiarazione e Definizione

Per usare una funzione, di solito sono necessari due passaggi: la **dichiarazione** (prototipo) e la **definizione** (implementazione).

Dichiarazione (Prototipo) La dichiarazione, o prototipo, informa il compilatore sull'esistenza di una funzione, specificando il suo nome, il tipo di dato che restituisce (tipo di ritorno) e i tipi di dati che accetta come input (parametri).

La sintassi è:

```
tipo_ritorno nome_funzione(tipo_param1, tipo_param2, ...);
```

I prototipi si inseriscono tipicamente all'inizio del file, dopo gli `#include`.

Definizione La definizione è il blocco di codice che costituisce la funzione vera e propria. Contiene le istruzioni che verranno eseguite quando la funzione è chiamata.

La sintassi è:

```
tipo_ritorno nome_funzione(tipo_param1 nome_param1, tipo_param2
   nome_param2, ...) {

// Corpo della funzione
```

```
    // ... istruzioni ...

    return valore_restituito; // Solo per funzioni non void
}
```

- `tipo_ritorno`: Il tipo di dato del valore che la funzione restituisce al chiamante. Se la funzione non restituisce nulla, si usa `void`.
- `nome_funzione`: Il nome della funzione.
- Parametri: Le variabili (es. `nome_param1`) che ricevono i valori passati alla funzione.
- `return`: L'istruzione che termina la funzione e (se non `void`) restituisce un valore.

7.2 Scope delle Variabili

Lo *scope* (visibilità) di una variabile definisce dove nel programma essa può essere utilizzata. Ogni variabile esiste solo all'interno del suo scope.

- **Variabili Locali**: Dichiarate all'interno di una funzione (inclusi i parametri). Sono visibili e utilizzabili **solo** all'interno di quella funzione. Vengono create quando la funzione inizia e distrutte (perdendo il loro valore) quando la funzione termina. Le variabili locali di una funzione (es. `main`) non sono visibili da un'altra funzione (es. `swap_errato`).
- **Variabili Globali**: Dichiarate all'esterno di qualsiasi funzione (di solito in cima al file). Sono visibili e modificabili da **qualsiasi** funzione nel file (dal punto di dichiarazione in poi). Il loro uso è generalmente sconsigliato perché rendono il programma difficile da comprendere e manutenere, creando dipendenze nascoste tra le funzioni.

7.3 Passaggio dei Parametri

Passaggio per Valore "Passare per valore" significa che, quando una funzione viene chiamata, il **valore** dell'argomento nel chiamante (es. `main`) viene **copiato** in una *nuova* variabile: il parametro formale della funzione chiamata.

La funzione lavora quindi esclusivamente su questa **copia locale**. Qualsiasi modifica fatta a questa copia *all'interno* della funzione **non** ha alcun effetto sulla variabile originale nel chiamante.

Ad esempio, un'operazione di scambio tra due varabili effettuata con il passaggio per valore non avrà effetto al di fuori della funzione:

```
// 'a' e 'b' sono copie locali, esistono solo qui dentro.
void swap_errato(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
    // Qui 'a' (la copia) e 'b' (la copia) sono state scambiate.
    // Le variabili originali 'x' e 'y' del main non vengono
       toccate.
}

void main() {
    int x = 10;
    int y = 20;

    // Il valore di x (10) viene copiato in 'a'
    // Il valore di y (20) viene copiato in 'b'
    swap_errato(x, y);

    // Al termine di swap_errato, 'a' e 'b' vengono distrutte.
    // 'x' e 'y' non sono mai state modificate.
    // Il risultato della printf rimane ancora "10, 20"
    printf("%d, %d", x, y);
}
```

Passaggio per Riferimento Cosa fare se vogliamo che una funzione modifichi le variabili del chiamante? Possiamo utilizzare **l'indirizzo di memoria** della variabile come parametro formale.

Il meccanismo è sempre un passaggio "per valore", ma il valore che passiamo non è il valore della variabile (es. 10), ma il valore del suo indirizzo (es. 0x7FFF...).

- Nel chiamante (`main`): Usiamo l'operatore `&` per ottenere l'indirizzo della variabile (es. `&x`).
- Nella funzione: Il parametro formale deve essere di tipo **puntatore** (dichiarato con `*`) per poter memorizzare un indirizzo.
- Nella funzione: Per accedere o modificare il valore *contenuto* a quell'indirizzo, usiamo l'operatore di "dereferenziazione" (`*`).

Possiamo ora effettuare uno scambio tra variabili che verrà propagato anche nel main:

```
void swap(int *a, int *b) {
    int temp;
```

```
    // *a significa "vai all'indirizzo contenuto in a"
    // e prendi il valore (10).
    temp = *a;

    // *b significa "vai all'indirizzo 'b' (a 'y')"
    // e prendi il valore (20).
    // *a = ... significa "vai all'indirizzo 'a' (a 'x') e
    // scrivi il nuovo valore (20).
    *a = *b;

    // *b = ... significa "vai all'indirizzo 'b' (a 'y') e
    // scrivi il valore di temp (10).
    *b = temp;
}

void main() {
    int x = 10;
    int y = 20;

    // Passo l'indirizzo di x
    // Passo l'indirizzo di y
    swap(&x, &y);

    // La funzione ha modificato direttamente i valori
    // alle locazioni di memoria di x e y.
    // Ora x vale 20 e y vale 10.
    printf("%d, %d", x, y);
}
```

7.4 Funzioni Ricorsive

Le funzioni ricorsive sono funzioni che si richiamano direttamente o indirettamente. La ricorsione è utile per risolvere problemi che possono essere suddivisi in sottoproblemi simili. Per risolvere un problema attraverso la programmazione ricorsiva sono necessari alcuni elementi:

- Caso base: caso elementare del problema che può essere risolto immediatamente.
- Passo ricorsivo: chiamata ricorsiva per risolvere uno o più problemi più semplici.
- Costruzione della soluzione: costruzione della soluzione sulla base del risultato delle chiamate ricorsive

Durante la costruzione di funzioni ricorsive è fondamentale aggiungere una condizione di terminazione che ci assicurerà che prima o poi la catena di chiamate a funzioni termini.

Esempio: Calcolo del Fattoriale. Il fattoriale di un numero n ($n!$) può essere calcolato ricorsivamente come:

$$n! = \begin{cases} 1 & \text{se } n = 0, \\ n \cdot (n-1)! & \text{se } n > 0. \end{cases}$$

```
int fattoriale(int n) {
    if (n == 0)         // Caso base
        return 1;
    else                // Caso ricorsivo
        return n * fattoriale(n - 1);
}
```

7.5 Esercizi

Esercizio 7.1

Scrivere un programma che legga una sequenza di numeri interi positivi, terminata dal valore 0. Il programma deve calcolare, il numero totale di valori inseriti (escluso lo 0), la somma di tutti i numeri, il massimo e il minimo valore inserito e la media dei valori inseriti.

Scrivere le funzioni:

- leggiNumero(): che legge e restituisce un numero naturale
- massimo(): restituisce il maggiore tra due numeri
- minimo(): restituisce il minore tra due numeri
- calcolaMedia(): restituisce la media dei numeri letti

Esercizio 7.2

Scrivere un programma che prende in input due numeri naturali $n1$ e $n2$ e verifica se sono amici Due numeri si dicono amici quando la somma dei divisori propri di $n1$ è uguale a $n2$ e viceversa.

- leggiNumero(): che legge e restituisce un numero naturale
- numeroAmico(): che riceve in ingresso due numeri naturali e restituisce vero
- sommaDivisori(): che riceve in ingresso un numero naturale e restituisce la somma dei suoi divisori (eccetto se stesso)

Esercizio 7.3

Scrivere un programma che chiede all'utente di inserire un numero naturale. Il programma verifica che il numero sia perfetto. Un numero si dice perfetto quando è uguale all somma di tutti i suoi divisori (eccetto se stesso).

Scrivere le funzioni:

- leggiNumero(): che legge e restituisce un numero naturale
- numeroPerfetto(): che riceve in ingresso un numero naturale e restituisce vero
- sommaDivisori(): che riceve in ingresso un numero naturale e restituisce la somma dei suoi divisori (eccetto se stesso)

Esercizio 7.4

Scrivere una funzione che riceve un array di dimensione N e un intero k e calcola la lunghezza della sequenza piu' lunga di interi consecutivi distanti tra loro esattamente k.

Esempio: $k = 3$, sequenza: 1 2 5 8 1 $\rightarrow$ 2

Esercizio 7.5

Si realizzi un programma che legga da input un numero non noto a priori di interi (al più 10) terminati da 0. Tale sequenza potrà evantualmente contenere numeri ripetuti. A tal fine, si realizzi una funziona `leggi(int v[], int dim)` che riceva in ingresso un vettore (vuoto) e la sua dimensione fisica, e restituisca la dimensione logica.

Si chieda poi l'inserimento di un valore da cercare nell'array. Si realizzi una funzione `search(int v[], int dim, int k)` che stampi a video tutti gli indici relativi alle posizioni in cui il numero k sia presente nell'array.

Esercizio 7.6

Creare un programma che legga da input una sequenza di interi, di lunghezza non nota a priori (al più 10), e terminata da 0. A tal scopo, si realizzi una funzione che riceva come parametri di ingresso un vettore e la sua dimensione fisica, e restituisca la dimensione logica del vettore. Tale funzione si deve fare carico della fase di lettura e riempimento dell'array. La sequenza può contenere numeri ripetuti.

Si realizzi una funzione che, ricevuti in ingresso il primo vettore con la sua dimensione logica, ed un secondo vettore con la sua dimensione fisica, memorizzi nel secondo vettore tutti gli elementi del primo, ma senza ripetizioni. La funzione restituisca la dimensione logica del secondo vettore.

Si realizzi un main che invochi le funzioni, e che stampi a video l'elenco degli elementi non ripetuti

Esercizio 7.7

Scrivere un programma che legge una stringa senza spazi e un carattere e stampa la stringa inserita senza il carattere inserito. Scrivere le funzioni:

- leggiCarattere(): che legge e restituisce un carattere
- leggiStringa(): che legge e restituisce una stringa
- rimuoviCarattere(): che prende in input una stringa e un carattere e restituisce la stringa senza il carattere inserito

Esercizio 7.8

Scrivere un programma che legge una matrice 3x4 ed effettua uno shift per righe verso il basso e stampa il risultato. L'ultima riga prende la posizione della prima riga. Scrivere le funzioni:

- leggiArray(): che legge e restituisce un array di 4 elementi
- stampaArray(): che stampa un array di 4 elementi
- copiaArray(): che copia un array di 4 elementi in un altro array di 4 elementi
- leggiMatrice(): che legge e restituisce una matrice 3x4
- stampaMatrice(): che stampa una matrice 3x4
- shiftMatrice(): che effettua lo shift della matrice verso il basso

Esercizio 7.9

Si scriva una funzione che riceva due varibili (base ed esponente) per valore, e due variabili di output (potenza e radice) passate per riferimento. La funzione deve calcolare la potenza (base elevata ad esponente) e la radice (radice ennesima di base) e restituirle tramite le variabili di output.

Esercizio 7.10

Si scriva un programma che gestisca un punto nel piano cartesiano. Si definisca una struttura `Punto` che rappresenti un punto nel piano. Si implementino le seguenti funzionalità:

- Una funzione `trasli` che riceve un punto e due valori `dx` e `dy` e trasla il punto di tali valori.
- Una funzione `baricentro` che riceve un insieme di punti e restituisce il punto baricentro.

Esercizio 7.11

Si scrivano le versioni ricorsiva ed iterativa di una funzione `double f(double a, int n)` che calcoli il seguente valore:

$$\sum_{i=1}^{n} a - \frac{i}{a}$$

Esercizio 7.12

Si scriva un programma che inverta le cifre di un numero intero N usando una funzione apposita. Si realizzi sia una versione ricorsiva che una versione iterativa.

Esempio: se l'input è 1234, l'output deve essere 4321.

Esercizio 7.13

Si scriva una funzione ricorsiva che inverta una stringa data in ingresso.

Esercizio 7.14

La *torre di Hanoi* è un famoso rompicapo matematico composto da tre aste `"A"`, `"B"` e `"C"` e un numero `N` di dischi di grandezza decrescente. Il rompicapo inizia con i dischi impilati in ordine crescente di dimensione sulla prima asta (`A`), il disco più piccolo in cima. L'obiettivo del gioco è spostare l'intera pila di dischi dalla prima asta (`A`) alla terza asta (`C`), rispettando le seguenti regole:

- Si può spostare solo un disco alla volta.
- Non si può posizionare un disco più grande sopra uno più piccolo.

Si scriva una funzione che risolva il problema in modo ricorsivo per un numero `N` di dischi dato in input, stampando ad ogni mossa il disco che viene spostato, l'asta di partenza e l'asta di destinazione.

Esempio: Per `N = 3`, l'output dovrebbe essere:

```
Sposta il disco 1 da A a C
Sposta il disco 2 da A a B
Sposta il disco 1 da C a B
```

```
Sposta il disco 3 da A a C
Sposta il disco 1 da B a A
Sposta il disco 2 da B a C
Sposta il disco 1 da A a C
```

Esercizio 7.15

Si scriva un programma per la gestione degli atleti che partecipano a una gara di triathlon. Ogni atleta è caratterizzato da un nome (stringa di massimo 50 caratteri), da un identificativo e un tempo totale di gara (in ore, minuti e secondi). Si implementino le seguenti funzionalità:

- Definizione delle strutture dati necessarie per rappresentare un atleta e una lista di atleti.
- Scrivere una funzione `somma_tempo` che riceve in input due tempi di gara e restituisce la somma dei due tempi.
- Scrivere una funzione `verifica_tempo_minore` che ricevuti due tempi di gara restituisce 1 se la prima variabile è minore o uguale alla seconda, 0 altrimenti.
- Scrivere una funzione `calcola_tempo_totale` che riceve in input un atleta e calcola il tempo totale di tutta la competizione.
- Scrivere una funzione `stampa_miglior_tempo` che riceve in input un'array di atleti e la sua dimensione e stampa a schermo l'id e il cognome dell'atleta con il tempo tempo totale migliore.

Soluzioni

Soluzione dell'esercizio 7.1

```c
#include <stdio.h>

/* Prototipi */
int leggiNumero();
int massimo(int a, int b);
int minimo(int a, int b);
float media(int somma, int count);

int main() {
    int numero;
    int count = 0;
    int somma = 0;
    int max, min;
    float m;

    printf("Inserisci numeri positivi (0 per terminare):\n");
    numero = leggiNumero();

    /* Gestione caso nessun numero */
    if (numero == 0) {
        printf("Nessun numero inserito.\n");
        return 0;
    }

    /* Inizializzo con il primo numero */
    max = numero;
    min = numero;
    somma = numero;
    count = 1;

    /* Ciclo di lettura successivi */
    numero = leggiNumero();
    while (numero != 0) {
        somma += numero;
        count++;

        max = massimo(max, numero);
        min = minimo(min, numero);

        numero = leggiNumero();
    }

    m = media(somma, count);

    printf("\nHai inserito %d numeri.\n", count);
    printf("Somma = %d\n", somma);
    printf("Massimo = %d\n", max);
    printf("Minimo = %d\n", min);
    printf("Media = %.2f\n", m);

    return 0;
}

/* Legge un numero da tastiera */
int leggiNumero() {
    int n;
```

```
    printf("Numero: ");
    do{
        scanf("%d", &n);
    }while(n < 0);
    return n;
}

/* Restituisce il maggiore tra due numeri */
int massimo(int a, int b) {
    if (a > b)
        return a;
    else
        return b;
}

/* Restituisce il minore tra due numeri */
int minimo(int a, int b) {
    if (a < b)
        return a;
    else
        return b;
}

/* Calcola la media */
float media(int somma, int count) {
    return (float)somma / (float)count;
}
```

Soluzione dell'esercizio 7.2

```
#include <stdio.h>

int leggiNumeroNaturale();
int sommaDivisori(int);
int numeriAmici(int, int);

int main(){
    // Variabili
    int num1, num2, flag;

    // Leggi il numero
    num1 = leggiNumeroNaturale();
    num2 = leggiNumeroNaturale();

    // Verifica se il numero è perfetto
    flag = numeriAmici(num1, num2);

    if(flag) printf("%d e %d amici\n", num1, num2);
    else printf("%d e %d non amici\n", num1, num2);

    return 0;
}

int leggiNumeroNaturale(){
    int numero;

    do {
        printf("Inserisci un numero: ");
        scanf("%d", &numero);
    } while(numero < 0);
```

```
    return numero;
}

int sommaDivisori(int n){
    int i, somma=0;

    for(i=1; i<=n; i++){
        if(n % i == 0) somma += i;
    }

    return somma;
}

int numeriAmici(int n1, int n2){
    int somma1, somma2;

    somma1 = sommaDivisori(n1) - n1;
    somma2 = sommaDivisori(n2) - n2;

    return somma1 == n2 && somma2 == n1;
}
```

Soluzione dell'esercizio 7.3

```
#include <stdio.h>

int leggiNumeroNaturale();
int sommaDivisori(int);
int numeroPerfetto(int);

int main(){
    // Variabili
    int num, flag;

    // Leggi il numero
    num = leggiNumeroNaturale();

    // Verifica se il numero è perfetto
    flag = numeroPerfetto(num);

    if(flag) printf("%d perfetto\n", num);
    else printf("%d non perfetto\n", num);

    return 0;
}

int leggiNumeroNaturale(){
    int numero;

    do {
        printf("Inserisci un numero: ");
        scanf("%d", &numero);
    } while(numero < 0);

    return numero;
}

int sommaDivisori(int n){
    int i, somma=0;
```

```
    for(i=1; i<=n; i++){
        if(n % i == 0) somma += i;
    }

    return somma;
}

int numeroPerfetto(int numero){
    int somma;

    somma = sommaDivisori(numero);

    return somma - numero == numero;
}
```

Soluzione dell'esercizio 7.4

```
#include<stdio.h>
#define N 6

void read_array(int []);
int sequenze(int [], int);

int main(){
  int v[N];
  int k,res;

  read_array(v);
  printf("Introdurre k:\n");
  scanf("%d",&k);

  res = sequenze(v,k);
  printf("numero di seq= %d\n",res);

  return 0;
}

void read_array(int v[]){
  int i;
  printf("Introdurre array di dimensione %d:\n", N);
  for (i=0; i<N; i++){
    scanf("%d",&v[i]);
  }
}

int sequenze(int v[], int k){
  //c: lunghezza della sequenza corrente
  //c_max: lunghezza della sequenza massima incontrata
  int i,c=1,c_max=1;

  for(i=0; i<N-1; i++){
    if(v[i+1]==v[i]+k || v[i+1]==v[i]-k) {
      c++;
      if(c_max < c)
        c_max = c;
    } else {
      c=1;
    }
  }
```

```
  return c_max;
}
```

Soluzione dell'esercizio 7.5

```
#include <stdio.h>
#include <stdlib.h>
#define DIM 10

int leggi(int vet[], int dim) {
    int size = 0, num;
    do {
        printf("Inserisci un numero: ");
        scanf("%d", &num);
        if (num!=0 && size<dim) {
            vet[size] = num;
            size++;
        }
    } while (num!=0 && size<dim);

    return size;
}

void search(int v[], int dim, int k) {
    int found = 0;
    for (int i = 0; i < dim; i++) {
        if (v[i] == k) {
            printf("Trovato %d all'indice %d\n", k, i);
            found = 1;
        }
    }
    if (!found) {
        printf("%d non trovato\n", k);
    }
}

int main() {
    int vettore[DIM];
    int size, k;

    size = leggi(vettore, DIM);

    printf("Inserisci il numero da cercare: ");
    scanf("%d", &k);

    search(vettore, size, k);

    return 0;
}
```

Soluzione dell'esercizio 7.6

```
#include <stdio.h>
#include <stdlib.h>
#define DIM 10

int leggi(int vet[], int dim) {
```

```
    int size = 0;
    int num;
    do {
        printf("Inserisci un numero: ");
        scanf("%d", &num);
        if (num!=0 && size<dim) {
            vet[size] = num;
            size++;
        }
    } while (num!=0 && size<dim);

    return size;
}

int elimina_ripetuti(int values[], int dim_v, int single[], int dim_s) {
    int size_s = 0;
    int i, j, trovato;

    for (i=0; i < dim_v && size_s < dim_s; i++) {
        trovato = 0;
        for (j=0; j<size_s && !trovato; j++) {
            if (values[i] == single[j])
                trovato = 1;
        }
        if (!trovato) {
            single[size_s] = values[i];
            size_s++;
        }
    }
    return size_s;
}

int main() {
    int v1[DIM], v2[DIM];
    int dim1 = leggi(v1, DIM);
    int dim2 = elimina_ripetizioni(v1, dim1, v2, DIM);

    printf("Elementi non ripetuti:\n");
    for (int i = 0; i < dim2; i++) {
        printf("%d\n", v2[i]);
    }

    return 0;
}
```

Soluzione dell'esercizio 7.7

```
// Librerie
#include <stdio.h>
#include <string.h>

// Costanti
#define LEN 100

// Prototipi
char leggiCarattere();
void leggiStringa(char []);
void rimuoviCarattere(char [], char [], char);

// Main
```

```
int main(){
    // Variabili
    char src[LEN], dest[LEN], da_rimuovere;

    // Leggi stringa
    leggiStringa(src);

    // Leggi Carattere
    da_rimuovere = leggiCarattere();

    // Rimuovi il carattere dalla stringa
    rimuoviCarattere(src, dest, da_rimuovere);

    // Stampa la stringa dst
    printf("%s senza %c = %s\n", src, da_rimuovere, dest);

    return 0;
}

// Funzioni
char leggiCarattere(){
    char c;
    printf("Inserisci carattere: ");
    scanf("%c", &c);
    scanf("%*c");
    return c;
}

void leggiStringa(char stringa[]){
    printf("Inserisci stringa: ");
    scanf("%s", stringa);
    fflush(stdin);
    return;
}

void rimuoviCarattere(char src[], char dest[], char da_rimuovere){
    // Variabili
    int len_src, i, len_dest;

    // Inizializzo Lunghezze
    len_src = strlen(src);
    len_dest = 0;

    // Scorro src per copiare senza buchi in dest
    for(i=0; i<len_src; i++){
        if(src[i] != da_rimuovere){
            dest[len_dest] = src[i];
            len_dest++;
        }
    }

    // Aggiungo carattere di terminazione
    dest[len_dest] = '\0';

    return;
}
```

Soluzione dell'esercizio 7.8

```
// Librerie
```

```c
#include <stdio.h>

// Costanti
#define R 3
#define C 4

// Prototipi
void leggiArray(int [], int);
void leggiMatrice(int [][C], int, int);
void stampaArray(int [], int);
void stampaMatrice(int [][C], int, int);
void copiaArray(int [], int [], int);
void shiftMatrice(int [][C], int [][C], int, int);

// Main
int main(){
    int src[R][C], dest[R][C];

    leggiMatrice(src, R, C);
    shiftMatrice(dest, src, R, C);

    printf("\nMatrice originale\n");
    stampaMatrice(src, R, C);

    printf("\nMatrice con shift\n");
    stampaMatrice(dest, R, C);

    return 0;
}

// Funzioni
void leggiArray(int v[], int n_colonne){
    int i;
    for(i = 0; i < n_colonne; i++){
        printf("Inserisci elemento in posizione %d: ", i);
        scanf("%d", &v[i]);
    }
    return;
}

void leggiMatrice(int m[][C], int n_righe, int n_colonne){
    int i;
    for(i = 0; i < n_righe; i++){
        printf("Inserisci riga %d\n", i);
        leggiArray(m[i], n_colonne);
        printf("\n");
    }
    return;
}

void stampaArray(int v[], int n_colonne){
    int i;

    printf("[");
    for(i = 0; i < n_colonne; i++){
        printf("%d", v[i]);
        if (i < n_colonne - 1) printf(", ");
    }
    printf("]\n");

    return;
}
```

```
void stampaMatrice(int m[][C], int n_righe, int n_colonne){
    int i;
    for(i = 0; i < n_righe; i++){
        stampaArray(m[i], n_colonne);
    }
    return;
}

void copiaArray(int dest[], int src[], int n_colonne){
    int i;
    for(i = 0; i < n_colonne; i++){
        dest[i] = src[i];
    }
    return;
}

void shiftMatrice(int dest[][C], int src[][C], int n_righe, int n_colonne){
    int i;

    for(i = 1; i < n_righe; i++){
        copiaArray(dest[i], src[i-1], n_colonne);
    }

    copiaArray(dest[0], src[n_righe-1], n_colonne);
    return;
}
```

Soluzione dell'esercizio 7.9

```
#include <stdio.h>
#include <math.h>

/* Funzione che modifica le variabili potenza e radice per riferimento */
void calcola(double base, double exp, double *potenza, double *radice)
{
    /* Uso delloperatore * per accedere ai valori puntati */
    *potenza = pow(base, exp);

    if (base >= 0.0)
        *radice = sqrt(base);
    else
        *radice = -1.0; /* valore sentinella per base negativa */
}

int main()
{
    double b, e;
    double p, r;

    printf("Inserisci base ed esponente: ");
    scanf("%lf %lf", &b, &e);

    /* Passaggio per riferimento con loperatore & */
    calcola(b, e, &p, &r);

    printf("\nRisultati calcolati tramite passaggio per riferimento:\n");
    printf("Base = %.3f, Esponente = %.3f\n", b, e);
    printf("Potenza = %.3f\n", p);
    if (r >= 0)
```

```
        printf("Radice quadrata = %.3f\n", r);
    else
        printf("Radice quadrata: impossibile (base negativa)\n");

    return 0;
}
```

Soluzione dell'esercizio 7.10

```
#include <stdio.h>

typedef struct{
    double x;
    double y;
} Punto;

/* Trasla un punto di (dx, dy)  modifica per riferimento */
void trasla(Punto *p, double dx, double dy)
{
    p->x += dx;
    p->y += dy;
}

/* Calcola il baricentro di n punti (restituito per riferimento) */
Punto baricentro(Punto *punti, int n)
{
    int i;
    Punto ris;
    ris.x = 0.0;
    ris.y = 0.0;

    for (i = 0; i < n; i++) {
        ris.x += punti[i].x;
        ris.y += punti[i].y;
    }

    ris.x /= n;
    ris.y /= n;

    return ris;
}

int main(void)
{
    Punto punti[3];
    Punto centro;
    int i;
    double dx, dy;

    printf("Inserisci le coordinate di 3 punti (x y):\n");
    for (i = 0; i < 3; i++)
        scanf("%lf %lf", &punti[i].x, &punti[i].y);

    printf("Vettore di traslazione (dx dy): ");
    scanf("%lf %lf", &dx, &dy);

    for (i = 0; i < 3; i++)
        trasla(&punti[i], dx, dy);  /* passaggio per riferimento */

    centro = baricentro(punti, 3);  /* risultato via valore di ritorno */
```

```
    printf("Punti traslati:\n");
    for (i = 0; i < 3; i++)
        printf("(%.2f, %.2f)\n", punti[i].x, punti[i].y);

    printf("Baricentro: (%.2f, %.2f)\n", centro.x, centro.y);

    return 0;
}
```

Soluzione dell'esercizio 7.11

```
#include <stdio.h>

/* Versione ricorsiva */
double f_ric(double a, int n) {
    if (n == 1)
        return a - 1 / a;  /* Caso base */
    else
        return (a - (n / a)) + f_ric(a, n - 1);
}

/* Versione iterativa */
double f_iter(double a, int n) {
    double somma = 0.0;
    int i;
    for (i = 1; i <= n; i++)
        somma += a - (i / a);
    return somma;
}

int main(void) {
    double a = 2.0;
    int n = 5;

    printf("Versione ricorsiva: f(%.2f, %d) = %.4f\n", a, n, f_ric(a, n));
    printf("Versione iterativa: f(%.2f, %d) = %.4f\n", a, n, f_iter(a, n));

    return 0;
}
```

Soluzione dell'esercizio 7.12

```
#include <stdio.h>

/* Versione ricorsiva */
int inverti_ric(int n, int rev) {
    if (n == 0)
        return rev; /* Caso base: nessuna cifra rimasta */
    else
        return inverti_ric(n / 10, rev * 10 + (n % 10));
}

/* Funzione di comodo per chiamare la ricorsiva */
int inverti_ricorsivo(int n) {
    return inverti_ric(n, 0);
}
```

```c
/* Versione iterativa */
int inverti_iterativo(int n) {
    int rev = 0;
    while (n != 0) {
        rev = rev * 10 + (n % 10);
        n = n / 10;
    }
    return rev;
}

int main(void) {
    int n;

    printf("Inserisci un numero intero: ");
    scanf("%d", &n);

    printf("Versione ricorsiva: %d\n", inverti_ricorsivo(n));
    printf("Versione iterativa: %d\n", inverti_iterativo(n));

    return 0;
}
```

Soluzione dell'esercizio 7.13

```c
// Librerie
#include <stdio.h>
#include <string.h>

#define LEN 100

// Prototipi
void invertiStringa(char [], char [], int*);

int main(){
    char src[LEN] = "ciao mamma!", dest[LEN];
    int len = 0;

    invertiStringa(src, dest+strlen(src)-1, &len);

    printf("Stringa originale: %s\n", src);
    printf("Stringa invertita: %s\n", dest);

    return 0;
}

// Funzioni
void invertiStringa(char src[], char dest[], int * og_len){
    if(src[0] != '\0'){
        // stringa sorgente non è finita
        // copio nelle attuali celle di memoria
        dest[0] = src[0];
        // incremento il counter di lettere copiate
        (*og_len) = (*og_len) + 1;
        // chiamo ricorsivamente cambiando i contatori
        invertiStringa(src+1, dest-1, og_len);
    }
    else{
        // dest sarà una cella prima dell'inizio e devo puntare alla fine
        dest[*og_len+1] = src[0];
    }
```

```
}
```

Soluzione dell'esercizio 7.14

```
#include <stdio.h>

/*
 * n  = numero di dischi
 * from = asta di partenza
 * to   = asta di destinazione
 * aux  = asta di appoggio
 */

void hanoi(int n, char from, char to, char aux) {
    if (n == 1) {
        /* Caso base: sposta un solo disco */
        printf("Sposta il disco 1 da %c a %c\n", from, to);
    } else {
        /* Sposta n-1 dischi dalla sorgente allasta di appoggio */
        hanoi(n - 1, from, aux, to);

        /* Sposta il disco più grande dalla sorgente alla destinazione */
        printf("Sposta il disco %d da %c a %c\n", n, from, to);

        /* Sposta n-1 dischi dallasta di appoggio alla destinazione */
        hanoi(n - 1, aux, to, from);
    }
}

int main() {
    int n;

    printf("Inserisci il numero di dischi: ");
    scanf("%d", &n);

    printf("\nSoluzione per %d dischi:\n", n);
    hanoi(n, 'A', 'C', 'B');

    return 0;
}
```

Soluzione dell'esercizio 7.15

```
#include <stdio.h>
#include <string.h>

// Definizione delle strutture dati
typedef struct {
    int ore;
    int minuti;
    int secondi;
} Tempo;

typedef struct {
    int id;
    char nome[51];  /* fino a 50 caratteri + terminatore */
    Tempo nuoto;
    Tempo bici;
```

```
    Tempo corsa;
    Tempo totale;
} Atleta;


Tempo somma_tempo(Tempo t1, Tempo t2) {
    Tempo risultato;
    risultato.secondi = t1.secondi + t2.secondi;
    risultato.minuti  = t1.minuti + t2.minuti + risultato.secondi / 60;
    risultato.ore     = t1.ore + t2.ore + risultato.minuti / 60;
    risultato.secondi = risultato.secondi % 60;
    risultato.minuti  = risultato.minuti % 60;
    return risultato;
}

int verifica_tempo_minore(Tempo t1, Tempo t2) {
    if (t1.ore < t2.ore)
        return 1;
    else if (t1.ore > t2.ore)
        return 0;

    if (t1.minuti < t2.minuti)
        return 1;
    else if (t1.minuti > t2.minuti)
        return 0;

    if (t1.secondi <= t2.secondi)
        return 1;
    else
        return 0;
}

Tempo calcola_tempo_totale(Atleta a) {
    Tempo temp;
    temp = somma_tempo(a.nuoto, a.bici);
    temp = somma_tempo(temp, a.corsa);
    return temp;
}

void stampa_miglior_tempo(Atleta atleti[], int n) {
    int i, indice_migliore = 0;

    for (i = 1; i < n; i++) {
        if (!verifica_tempo_minore(atleti[indice_migliore].totale, atleti[i].
            totale)) {
            indice_migliore = i;
        }
    }

    printf("\nMiglior tempo:\n");
    printf("ID: %d, Nome: %s\n",
        atleti[indice_migliore].id,
        atleti[indice_migliore].nome);
    printf("Tempo totale: %02d:%02d:%02d\n",
        atleti[indice_migliore].totale.ore,
        atleti[indice_migliore].totale.minuti,
        atleti[indice_migliore].totale.secondi);
}

int main(void) {
    int n, i;
    Atleta atleti[100]; // dimensione massima
```

```
    printf("Inserisci il numero di atleti: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("\n--- Atleta %d ---\n", i + 1);
        atleti[i].id = i + 1;

        printf("Nome: ");
        scanf("%s", atleti[i].nome);

        printf("Tempo nuoto (h m s): ");
        scanf("%d %d %d", &atleti[i].nuoto.ore, &atleti[i].nuoto.minuti, &atleti[i
            ].nuoto.secondi);

        printf("Tempo bici (h m s): ");
        scanf("%d %d %d", &atleti[i].bici.ore, &atleti[i].bici.minuti, &atleti[i].
            bici.secondi);

        printf("Tempo corsa (h m s): ");
        scanf("%d %d %d", &atleti[i].corsa.ore, &atleti[i].corsa.minuti, &atleti[i
            ].corsa.secondi);

        atleti[i].totale = calcola_tempo_totale(atleti[i]);
    }

    stampa_miglior_tempo(atleti, n);

    return 0;
}
```